DATA STRUCTURES

Unit-III: Linked Lists & Sorting

Comprehensive Study Notes

Topic 1: Introduction to Linked List

1.1 What is a Linked List?

A **linked list** is a linear data structure where elements (called nodes) are stored at non-contiguous memory locations. Each node contains a data field and a pointer (link) to the next node in the sequence.

Advantages over Arrays

Feature	Array	Linked List
Memory allocation	Contiguous (fixed size)	Dynamic (grows/shrinks)
Insertion/Deletion	O(n) - shifting required	O(1) at head, O(n) at position
Memory utilization	Wastage if underused	Efficient, allocated per node
Random access	O(1) via index	O(n) - must traverse
Size	Fixed at compile time	Dynamic, grows at runtime

Representation in Memory

```
Node Structure:
+---------+--------+
|  DATA   |  NEXT  |
+---------+--------+

Memory Layout:
[1000] [1004] [1008] [1012]
+------+------+      +------+------+      +------+------+
|  10  | 1004 | ---> |  20  | 1008 | ---> |  30  | NULL |
+------+------+      +------+------+      +------+------+
 1000                 1004                 1008
 HEAD
```

Topic 2: Singly Linked List

2.1 Node Structure

```
struct Node {
    int data;
    struct Node* next;
};
```

2.2 Operations on Singly Linked List

Create a Node

```
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

Insert at Beginning

```
void insertAtBeginning(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *head;
    *head = newNode;
}
```

Insert at End

```
void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
```

```
    }
    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}
```

Insert at Position

```
void insertAtPosition(struct Node** head, int data, int pos) {
    struct Node* newNode = createNode(data);
    if (pos == 1) {
        newNode->next = *head;
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;
    if (temp == NULL) return;
    newNode->next = temp->next;
    temp->next = newNode;
}
```

Delete a Node

```
void deleteNode(struct Node** head, int key) {
    struct Node* temp = *head, *prev = NULL;
    if (temp != NULL && temp->data == key) {
        *head = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return;
    prev->next = temp->next;
    free(temp);
}
```

Display Linked List

```
void display(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

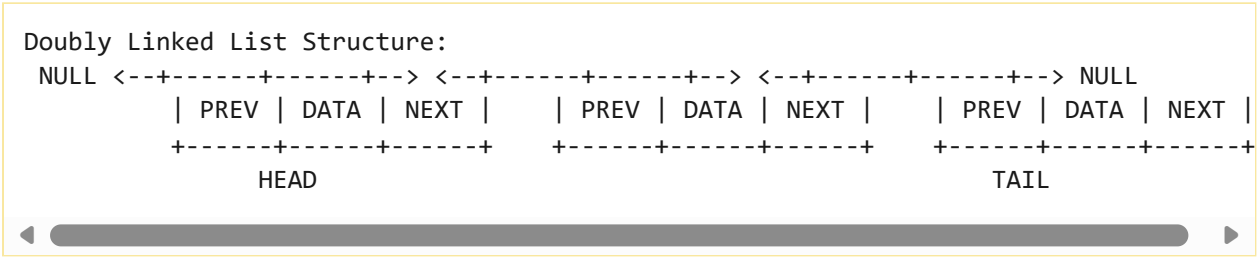
Search in Linked List

```
int search(struct Node* head, int key) {
    struct Node* temp = head;
    int pos = 0;
    while (temp != NULL) {
        if (temp->data == key)
            return pos;
        temp = temp->next;
        pos++;
    }
    return -1;
}
```

Topic 3: Doubly Linked List

3.1 Node Structure

```
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};
```



Insert at Beginning (Doubly)

```
void insertAtBeginning(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *head;
    if (*head != NULL)
        (*head)->prev = newNode;
    *head = newNode;
}
```

Insert at End (Doubly)

```
void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
```

```
    temp->next = newNode;
    newNode->prev = temp;
}
```

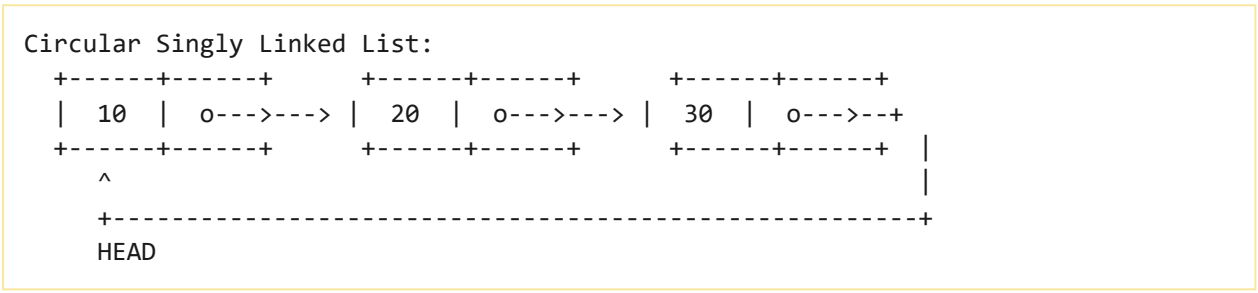
Delete a Node (Doubly)

```
void deleteNode(struct Node** head, int key) {
    struct Node* temp = *head;
    while (temp != NULL && temp->data != key)
        temp = temp->next;
    if (temp == NULL) return;
    if (temp->prev != NULL)
        temp->prev->next = temp->next;
    else
        *head = temp->next;
    if (temp->next != NULL)
        temp->next->prev = temp->prev;
    free(temp);
}
```

Topic 4: Circular Linked List

4.1 Concept

A **circular linked list** is a linked list where the last node points back to the first node (head), forming a circle. There is no NULL pointer at the end.



Traverse Circular Linked List

```
void display(struct Node* head) {
    if (head == NULL) return;
    struct Node* temp = head;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(back to head)\n");
}
```

Insert at Beginning (Circular)

```
void insertAtBeginning(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        newNode->next = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != *head)
        temp = temp->next;
```

```
    temp->next = newNode;
    newNode->next = *head;
    *head = newNode;
}
```

Topic 5: Applications of Linked Lists

Application	Description
Dynamic Memory Management	Free lists in OS memory allocation
Implementation of Stacks & Queues	Using linked lists for dynamic stack/queue
Polynomial Arithmetic	Representing and adding polynomials
Graph Adjacency Lists	Representing graph edges efficiently
Music/Video Playlists	Circular linked list for repeat play
Undo/Redo Operations	Doubly linked list for navigation
Hash Table Chaining	Collision resolution in hashing
File Systems	Allocating disk blocks

Topic 6: Insertion Sort

6.1 Algorithm

Insertion Sort builds the final sorted array one element at a time by repeatedly picking the next element and inserting it into its correct position among the previously sorted elements.

Algorithm

```
1. Start from the second element (index 1)
2. Pick the current element as key
3. Compare key with elements in the sorted portion (left side)
4. Shift all larger elements one position to the right
5. Insert key at the correct position
6. Repeat for all elements
```

```
Insertion Sort Example:
Array: [12, 11, 13, 5, 6]

Step 1: [11, 12, 13, 5, 6]   (insert 11 before 12)
Step 2: [11, 12, 13, 5, 6]   (13 already in place)
Step 3: [5, 11, 12, 13, 6]   (insert 5 at beginning)
Step 4: [5, 6, 11, 12, 13]   (insert 6 after 5)
Result: [5, 6, 11, 12, 13]
```

Insertion Sort Code

```
void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
```

```
        }
    }
```

Complexity

Case	Time Complexity
Best Case (already sorted)	O(n)
Average Case	$O(n^2)$
Worst Case (reverse sorted)	$O(n^2)$
Space Complexity	O(1)

Topic 7: Quick Sort

7.1 Algorithm

Quick Sort is a **divide and conquer** algorithm that selects a **pivot** element and partitions the array around it.

Algorithm

```
1. Choose a pivot element (usually last element)
2. Partition the array into two sub-arrays:
   - Elements less than pivot (left side)
   - Elements greater than pivot (right side)
3. Recursively sort the sub-arrays
```

Partition Algorithm

```
1. Set pivot = arr[high] (last element)
2. Initialize i = low - 1
3. For j = low to high - 1:
   a. If arr[j] < pivot, increment i and swap arr[i] with arr[j]
4. Swap arr[i+1] with arr[high] (place pivot in correct position)
5. Return i+1 (pivot index)
```

Quick Sort Code

```
#include <stdio.h>

void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    quickSort(arr, 0, n - 1);
    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Complexity

Case	Time Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	$O(n^2)$
Space Complexity	O(log n)

Topic 8: Sorting Algorithms Comparison

Algorithm	Best Case	Average Case	Worst Case	Space	Stable
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No

Questions

Short Answer Questions (2-5 Marks)

1. What is a linked list? How is it different from an array?
2. What are the advantages of linked lists over arrays?
3. Define node structure for a singly linked list.
4. What is a doubly linked list? How is it different from singly linked list?
5. What is a circular linked list?
6. What are the applications of linked lists?
7. Explain the algorithm of insertion sort.
8. Explain the algorithm of quick sort.
9. What is the time complexity of insertion sort?
10. What is the time complexity of quick sort?

Long Answer Questions (10 Marks)

1. Explain singly linked list with all operations (create, insert, delete, display, search).
2. Explain doubly linked list with insertion and deletion operations.
3. Explain circular linked list with insertion and traversal operations.
4. Explain insertion sort with algorithm, example, and code.
5. Explain quick sort with algorithm, example, and code.
6. Compare arrays and linked lists in detail.
7. Write a complete program to implement all operations on a singly linked list.

Objective Questions (1 Mark)

1. Linked list uses: (a) Contiguous memory (b) Non-contiguous memory (c) Both (d) None
   Answer: (b)
2. Each node in a singly linked list contains: (a) Data only (b) Data and next pointer (c) Data, prev, next (d) None
   Answer: (b)

3. Time complexity of insertion at beginning of linked list: (a) O(1) (b) O(n) (c) $O(n^2)$ (d) O(log n)
   Answer: (a)
4. Doubly linked list has how many pointers per node? (a) 1 (b) 2 (c) 3 (d) 0
   Answer: (b)
5. Insertion sort best case time complexity: (a) O(n) (b) $O(n^2)$ (c) O(log n) (d) O(n log n)
   Answer: (a)
6. Quick sort worst case time complexity: (a) O(n) (b) O(n log n) (c) $O(n^2)$ (d) O(log n)
   Answer: (c)